

Heliport

Guided migration from Dropwizard 3 to Helidon 4.5 and Talon

Helidon Dev2Dev · July 2026

OCI will be mandating Dropwizard migration to Helidon and Talon

Dropwizard service

- Dropwizard Application and Environment
- Jersey / JAX-RS resources and Servlet request/response types
- Guice modules, providers, and injector lookups
- Jetty manages the server lifecycle
- Guice providers construct OCI clients



Helidon 4.5 SE + Talon

- Helidon runtime discovery and WebServer
- Declarative HTTP endpoints
- Compile-time Service Registry
- JDK 25 and virtual threads
- Talon modules configure and inject OCI clients

Runtime responsibility moves. Before removing old wiring, Heliport must identify the Helidon or Talon component that now provides the same behavior.

450+ OCI teams need a faster path to Helidon

OCI leadership will be mandating Dropwizard-to-Helidon migration, accelerated by Dropwizard 3 end-of-life.

450+

teams affected
need a Helidon path

7–9 months

manual migration estimate
SPLAT fleet sizing

~2 hours / 4 months

automated migration / to production
automation compresses the code-conversion step

Benchmark proof

A completed migration changed substantial application code and passed every build gate.

175

files changed

9,737

insertions

3,034

deletions

PASS

compile · unit · install

+99%

8,117 → 16,221

requests per second / core

Modernization works best as an ordered sequence

1

Move to Java 25

Use the JDK migration skill first and verify that branch.

2

Run Heliport

Analyze, plan, migrate, validate, and report.

3

Resolve actions

The service team resolves the remaining migration issues reported by Heliport.

4

Review and ship

Inspect the diff and validation results, then push and open the normal PR.

Preferred input: a Java-25-ready Dropwizard 3 Maven repository. Heliport can make a best effort without that prerequisite, but the work is harder to isolate and review.

Heliport combines OpenRewrite recipes with Codex guidance

OpenRewrite applies repeatable transformations

- Maven and dependency convergence
- Framework annotations and runtime APIs
- Known provider, resource, lifecycle, and test patterns
- Repeatable, reviewable recipe execution

Codex guides the migration workflow

- Analyze the application and create a migration plan
- Run phases in the required order and resume safely
- Run validation and interpret failures
- Generate the migration report and remaining actions

Result: migrated source, validation results, and a clear list of work that still requires developer judgment.

Heliport uses repository facts instead of guessing



Heliport will

- record the files and code behind each decision
- preserve code that is still required
- report remaining developer work
- resume from saved migration state

Heliport will not

- rewrite business or security policy
- invent behavior for unsupported patterns
- report validation success when checks failed
- push the repository

Install Heliport as a global Codex skill

Clone the repository

Oracle DevOps
HLDN / heliport

Repository URL

<https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/heliport>

The checkout contains the skill, OpenRewrite recipes, guided workflow, report formats, and product tests.

Symlink the skill

```
mkdir -p ~/.codex/skills  
ln -s /abs/path/to/heliport \  
    ~/.codex/skills/heliport-migrate
```

Alternatively, locate the checkout:

```
export  
HELIPORT_SUPPORT_ROOT=/abs/path/to/heliport
```

Run Codex from the application workspace and start Heliport

```
cd /abs/path/to/dropwizard-app

codex -s workspace-write \
  --add-dir ~/.m2 \
  --add-dir /abs/path/to/dropwizard-app/.git \
  -c sandbox_workspace_write.network_access=true
```

Maven

The configured local repository must be writable.

Network

Maven and OpenRewrite may need missing artifacts and proxy settings.

Git

Write access to `.git` is required only for branch or commit options.

From that Codex session: invoke `$heliport-migrate`. Detailed options are in the appendix.

Heliport migrates the entire service, not only Java source

Build

Java 25 · BOMs · plugins · annotation processing

Generated contracts

OpenAPI APIs · models · fixtures

Runtime

Dropwizard · Guice · JAX-RS · Servlet

Platform

Talon · OCI clients · auth · TLS · metrics

Configuration

HOCON/YAML · source precedence · typed configuration

Tests

health checks · route tests · Panga · validation

A migration is complete only when every detected area is migrated, intentionally retained for compatibility, or listed as remaining developer work.

Establish the Helidon Maven model before runtime migration

Heliport configures

- Java 25 compiler and toolchain settings
- Helidon and OCI/Talon BOM imports
- Helidon runtime and feature dependencies
- service-registry annotation processing
- generated-source prerequisites
- logging implementation and plugin management

Heliport checks before cleanup

- explicit versions versus managed versions
- old Dropwizard, Jersey, Jetty, and Netty dependencies
- compatibility libraries still referenced by code
- plugin and enforcer conflicts
- feature modules not used by the application
- parent or multi-module dependency-management conflicts

Dependencies are removed only after their use is checked. A Netty or Jackson artifact remains when source, generated code, or tests still require it.

BOM imports centralize version convergence

Versions declared per dependency

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>${slf4j.version}</version>
</dependency>

<dependency>
  <groupId>com.oracle.oci.sdk</groupId>
  <artifactId>oci-java-sdk-identity</artifactId>
  <version>${oci.sdk.version}</version>
</dependency>
```

Versions managed by imported BOMs

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.helidon</groupId>
      <artifactId>helidon-dependencies</artifactId>
      <version>${helidon.version}</version>
      <type>pom</type><scope>import</scope>
    </dependency>
    <dependency>
      <groupId>com.oracle.helidon.oci</groupId>
      <artifactId>helidon-oci-dependencies</artifactId>
      <version>${helidon.oci.version}</version>
      <type>pom</type><scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Child modules declare what they use, not which version wins. Imported BOMs govern Helidon and Talon versions consistently across the reactor.

The root POM selects versions; modules declare dependencies

Reactor root: dependency management

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.helidon</groupId>
      <artifactId>helidon-dependencies</artifactId>
      <version>${helidon.version}</version>
      <type>pom</type><scope>import</scope>
    </dependency>
    <dependency>
      <groupId>com.oracle.helidon.oci</groupId>
      <artifactId>helidon-oci-dependencies</artifactId>
      <version>${helidon.oci.version}</version>
      <type>pom</type><scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Child module: direct dependencies

```
<dependencies>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
  </dependency>

  <dependency>
    <groupId>com.oracle.oci.sdk</groupId>
    <artifactId>oci-java-sdk-identity</artifactId>
  </dependency>
</dependencies>
```

Review rule: remove a child `<version>` only when `mvn help:effective-pom` confirms that the dependency is managed.

Child modules declare only the platform dependencies they use

Before: broad framework dependencies

```
<dependency>
  <groupId>io.dropwizard</groupId>
  <artifactId>dropwizard-core</artifactId>
</dependency>
<dependency>
  <groupId>io.dropwizard</groupId>
  <artifactId>dropwizard-auth</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
</dependency>
```

After: dependencies match module usage

```
<dependency>
  <groupId>io.helidon.webserver</groupId>
  <artifactId>helidon-webserver</artifactId>
</dependency>
<dependency>
  <groupId>io.helidon.http.media</groupId>
  <artifactId>helidon-http-media-jsonp</artifactId>
</dependency>
module orders.api {
  requires io.helidon.webserver;
  requires io.helidon.http.media.jsonp;
}
```

Dependencies follow source usage. Heliport removes a compatibility dependency only after source, generated code, and tests no longer require it.

Keep generated OpenAPI and handwritten code aligned

Heliport migrates together

- Maven generator plugin and configuration
- generated APIs and models
- handwritten implementations
- tests and fixtures

Heliport reports

- service-side generated sources that still expose JAX-RS, Servlet, or old resource-base APIs
- handwritten implementations that no longer match the generated Helidon contract
- compile or validation mismatches
- external specification inputs
- API behavior requiring a service-team decision

Do not hand-edit generated code. Heliport updates the generator configuration, regenerates the sources, and then checks handwritten implementations against the new contracts.

OpenAPI generation moves from JAX-RS bases to Helidon contracts

Before: Oracle JAX-RS generation

```
<plugin>
  <groupId>com.oracle.pic.commons.maven</groupId>
  <artifactId>oracle-swagger-maven-plugin</artifactId>
  <configuration>
    <language>oracle-java-jaxrs-service</language>

    <generateAbstractResources>true</generateAbstractResources>
  </configuration>
</plugin>
```

After: Helidon declarative generation

```
<plugin>
  <groupId>org.openapitools</groupId>
  <artifactId>openapi-generator-maven-
    plugin</artifactId>
  <dependencies>
    <dependency>
      <groupId>io.helidon.extensions.openapi-
        generator</groupId>
      <artifactId>helidon-extensions-openapi-
        generator</artifactId>
    </dependency>
  </dependencies>
  <configuration>
    <generatorName>helidon-declarative</generatorName>
    <output>${project.build.directory}/generated-
      sources/openapi</output>
    <addCompileSourceRoot>true</addCompileSourceRoot>
    <skipValidateSpec>true</skipValidateSpec>
  </configuration>
</plugin>
```

Heliport preserves the existing specification-validation step; `skipValidateSpec` prevents the generator from performing a second validation pass.

Handwritten resources implement the generated Helidon API

Before: JAX-RS base defines HTTP

```
final class WorkRequestResource
    extends AbstractWorkRequestBaseResource {

    public TaggedResponse<WorkRequest> get(...) {
        return TaggedResponse.ok(service.get(...));
    }
}
```

After: generated API defines the contract

```
@RestServer.Endpoint
@Service.Singleton
class WorkRequestResource
    implements WorkRequestsApi {

    @Override
    public WorkRequest get(...) {
        return service.get(...);
    }
}
```

Stable extension point: implement the generated `*Api` in handwritten code. Do not move business logic into generated `*Endpoint` classes.

Server-side JAX-RS resources become declarative Helidon endpoints

JAX-RS / Jersey

```
@Path("/v1")
public class ItemResource {
    @GET
    @Path("/items")
    public List<Item> list() {
        return service.list();
    }
}
```

Helidon declarative HTTP

```
@RestServer.Endpoint
@Service.Singleton
@Http.Path("/v1")
public class ItemResource {
    @Http.GET
    @Http.Path("/items")
    public List<Item> list() {
        return service.list();
    }
}
```

Looks familiar, runs differently. Helidon registers and serves the endpoint; no Jersey server runs in the migrated service.

JAX-RS request context becomes explicit Helidon input

Before: injected JAX-RS context

```
@GET
public Item get(
    @HeaderParam("opc-request-id") String
    requestId,
    @Context SecurityContext security) {
    return service.get(requestId,
        (Principal) security.getUserPrincipal());
}
```

After: Talon request and identity types

```
@Http.GET
@Identity.Authenticated
public Item get(
    OciRequestId requestId,
    Principal principal) {
    return service.get(
        requestId.upstreamHeaderValue(),
        principal);
}
```

Typed parameters replace request carriers only when the selected Talon integrations provide equivalent request-ID and identity behavior.

Response migration keeps observable HTTP behavior

Before: JAX-RS response builder

```
return Response.status(202)
    .header("opc-work-request-id", id)
    .type(MediaType.APPLICATION_JSON)
    .entity(summary)
    .build();
```

After: Helidon ServerResponse

```
response.status(Status.ACCEPTED_202)
    .header("opc-work-request-id", id)
    .header(HeaderNames.CONTENT_TYPE,
        MediaType.APPLICATION_JSON)
    .send(summary);
```

Preserve: status, headers, content type, body, empty sends, streaming, and statuses or headers required by the generated OpenAPI contract.

Dropwizard bootstrap becomes discovered Helidon services

Dropwizard registrations

```
class OrdersApplication extends
Application<OrdersConfig> {
    void run(OrdersConfig c, Environment e) {
        e.jersey().register(new SenderResource(c));
        e.lifecycle().manage(new QueueLifecycle());
        e.healthChecks().register("kiev", check);
        e.admin().addTask(new DeepcheckTask());
    }
}
```

Helidon startup and discovered services

```
public final class OrdersMain {
    public static void main(String[] args) {
        Main.main(args);
    }

    @RestServer.Endpoint
    @Service.Singleton
    class SenderResource { ... }

    @Service.Singleton
    class QueueLifecycle {
        @Service.PostConstruct
        void start() { ... }
    }
}
```

Heliport removes the Dropwizard `Application` only after every `Environment` registration has a Helidon replacement or is listed as remaining migration work.

Dropwizard lifecycle hooks become service lifecycle

Before: container-managed object

```
final class QueueLifecycle implements Managed {  
    @Override  
    public void start() {  
        consumer.start();  
    }  
  
    @Override  
    public void stop() {  
        consumer.close();  
    }  
}  
  
environment.lifecycle().manage(queue);
```

After: Helidon service lifecycle

```
@Service.Singleton  
final class QueueLifecycle {  
    @Service.PostConstruct  
    void start() {  
        consumer.start();  
    }  
  
    @Service.PreDestroy  
    void stop() {  
        consumer.close();  
    }  
}
```

Safe when lifecycle registration is unconditional. Conditional registration, startup ordering, or external framework lifecycle behavior remains migration work for the service team.

Health checks and admin tasks keep their behavior

Before: Dropwizard registries

```
environment.healthChecks()
    .register("store", new StoreHealthCheck(store));

environment.admin()
    .addTask(new DeepcheckTask(service));

class DeepcheckTask extends Task {
    void execute(Map<String, List<String>> p,
                PrintWriter out) { ... }
}
```

After: Helidon health and task endpoint

```
@Service.Singleton
final class StoreHealthCheck implements HealthCheck {
    @Override
    public HealthCheckResponse call() { ... }
}

@RestServer.Endpoint
@Service.Singleton
@Path("/tasks/deepcheck")
class DeepcheckTask {
    @Http.POST
    @Http.Produces(MediaType.TEXT_PLAIN_VALUE)
    String execute(ServerRequest request) { ... }
}
```

Heliport can preserve the old `execute(...)` body for direct tests while adding the production HTTP entry point.

Dropwizard configuration moves to the service that uses it

Before: Dropwizard configuration DTO

```
final class QueueConsumer {  
    QueueConsumer(OrdersConfig config) {  
        this.timeout = config.getQueueTimeout();  
    }  
}
```

After: service reads its configuration

```
@Service.Singleton  
final class QueueConsumer {  
    @Service.Inject  
    QueueConsumer(Config config) {  
        this.timeout = config.get("queue.timeout")  
            .as(Duration.class)  
            .get();  
    }  
}
```

Heliport preserves configuration source order, defaults, validation, secret lookup, and environment or system-property overrides.

Guice provider methods become Helidon supplier services

Guice provider

```
@Provides
@Singleton
@Named("clock")
Clock provideClock() {
    return Clock.systemUTC();
}
```

Helidon supplier

```
@Service.Singleton
@Service.Named("clock")
final class ClockSupplier
    implements Supplier<Clock> {
    private final Clock value =
        Clock.systemUTC();

    @Override
    public Clock get() {
        return value;
    }
}
```

What Heliport can migrate

Provider methods and named bindings can become service-registry suppliers when construction and scope are clear.

Behavior Heliport preserves

Binding name, singleton scope, dependencies, and intentional lazy initialization are carried into the target.

Guice constructor injection moves to Helidon Service Registry

Before: Guice binding

```
final class Sender {  
    private final Queue queue;  
  
    @Inject  
    Sender(@Named("outbound") Queue queue) {  
        this.queue = queue;  
    }  
}  
  
bind(Sender.class).in(Singleton.class);
```

After: Helidon service

```
@Service.Singleton  
final class Sender {  
    private final Queue queue;  
  
    @Service.Inject  
    Sender(  
        @Service.Named("outbound") Queue queue) {  
        this.queue = queue;  
    }  
}
```

Helidon generates service metadata at compile time; the application no longer needs a central binding for `Sender`.

A Guice module disappears only after every binding moves

Before: one module combines many concerns

```
install(new KievModule(config.getKiev()));
install(new
WorkflowClientModule(config.getWorkflow()));

bind(Sender.class).in(Singleton.class);
bind(Clock.class).toProvider(ClockProvider.class);

Multibinder.newSetBinder(binder(), Handler.class)
    .addBinding().to(RetryHandler.class);
```

After: Helidon services and Talon integrations

```
@Service.Singleton
final class Sender { ... }

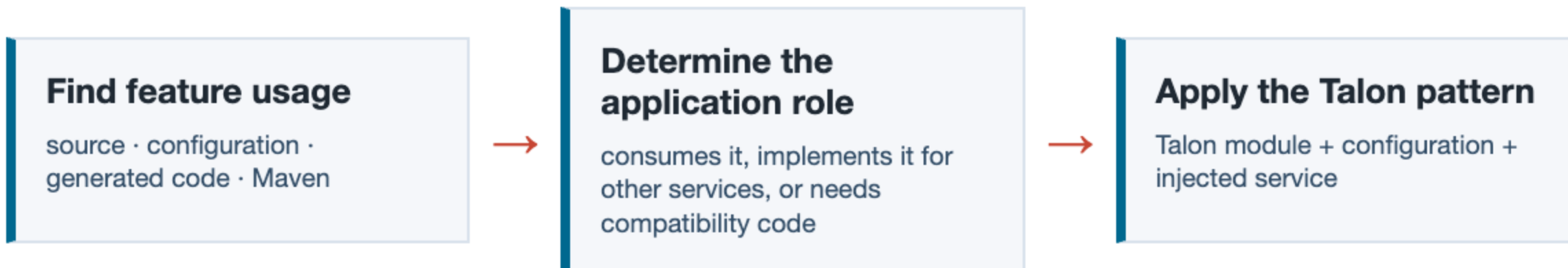
@Service.Singleton
final class ClockSupplier
    implements Supplier<Clock> { ... }

@Service.Singleton
final class RetryHandler
    implements Handler { ... }

// Talon modules provide Kiev and Workflow
// services through Service Registry.
```

Completion check: bindings, providers, qualifiers, runtime injector lookups, multibindings, and tests are migrated or listed as remaining developer work.

Heliport adds only the Talon integrations an application uses



Standard setup moves

Service endpoint, authentication, environment, TLS, and client creation can move into a Talon integration.

Service-specific behavior stays

Payloads, retry policy, authorization decisions, tenancy, custom signers, and error policy remain in the application.

Heliport does not add every Talon integration. A matching package name by itself is not enough.

Heliport migrates each OCI integration as a complete feature

Area	What Heliport includes
Maven	client libraries, provider APIs, auth modules, generated artifacts, and test dependencies
Java composition	Guice modules, <code>@Provides</code> methods, qualifiers, suppliers, and runtime injector lookups
Configuration	Dropwizard DTOs, YAML/HOCON keys, endpoints, timeouts, retry, auth, TLS, and environment values
Runtime use	injected clients, filters, annotations, transaction wrappers, request context, and SDK calls
Tests	in-memory providers, service overrides, fixture configuration, workflow drivers, and client doubles

Feature family: the Maven, Java, configuration, runtime, and test elements that implement one OCI integration. Heliport plans and migrates them together.

Identity: Jersey filters become generated Talon interceptors

Before: Dropwizard + JAX-RS

```
environment.jersey().register(authFilter);

@POST
@Path("/orders")
@AuthorizationPermission("ORDER_CREATE")
Order create(CreateOrder request,
              @Context SecurityContext security) {
    return service.create(request,
        (Principal) security.getUserPrincipal());
}

identity:
  serviceName: my-service
```

After: Talon Identity

```
@RestServer.Endpoint
@Http.Path("/orders")
class OrdersEndpoint {
    @Http.POST
    @AuthorizationPermission("ORDER_CREATE")
    Order create(@Http.Entity CreateOrder request,
                 Principal principal) {
        return service.create(request, principal);
    }
}

oci:
  identity:
    authentication:
      global-business-unit: "Cloud-Infra"
      team-name: "ExampleTeam"
      application-name: "MyService"
      use-instance-principal: true
    authorization:
      enabled: true
      service-name: "my-service"
```

`@AuthorizationPermission` activates generated authentication and authorization handling; Talon injects the authenticated OCI `Principal` directly.

Kiev: Guice-built stores become named Talon services

Before: Guice constructs the store

```
@Provides @Singleton @Named("orders-store")
MappedDataStore store(KievConfig c) {
    return new MappedDataStore(
        new DataStoreConfigBase(c), ...);
}

kiev:
  useKievInMemory: true
  storeName: orders-store
  appName: MyService
```

After: Talon constructs and injects the store

```
@Service.Singleton
class OrderStore {
    @Service.Inject
    OrderStore(@Service.Named("orders-store")
        MappedDataStore store) {
        bucket = store.getOrCreateBucket("orders", ...);
    }

    @KievTransaction(value = "orders-store",
        name = "order-put")
    Order put(Transaction tx, Order order) { ... }
}

oci:
  kiev:
    data-stores:
      - backend: "IN_MEMORY"
        store-name: orders-store
        app-name: MyService
        transaction-max-reads: 100
        transaction-max-writes: 100
```

Heliport carries each legacy store into `oci.kiev.data-stores` with its actual backend, locality, authentication, names, and transaction limits; `IN_MEMORY` is preserved here because the source selects it.

Metrics: Codahale and T2 configuration become Helidon + OCI publishing

Before: Dropwizard / Codahale

```
@GET
@Path("/hello/{name}")
@Timed(name = "personalized-greeting",
      absolute = true)
String greeting(@PathParam("name") String name) {
    return "Hello, " + name + "!";
}

metricsConfig:
  projectName: my-service
  metricsScopeName: my-service
  t2Config:
    fleet: my-fleet
    region: us-ashburn-1
```

After: Helidon metrics + Talon publisher

```
@Http.GET
@Path("/hello/{name}")
@Metrics.Timed(value = "personalized-greeting",
              absoluteName = true)
String greeting(@Http.PathParam("name") String name) {
    return "Hello, " + name + "!";
}

metrics:
  publishers:
    - type: oci
      project: my-service
      fleet: my-fleet
      region: us-ashburn-1
      metrics-scope-name: my-service
```

Heliport preserves the metric name and recognized publisher values while changing the recording API and configuration shape. Filters, dimensions, and reporting policy remain explicit.

A feature family is complete only when every part has migrated

Incomplete integration

- Talon module added while the Guice provider still constructs the client
- new config keys written while Java still reads the Dropwizard DTO
- binding removed before named consumers migrate
- legacy dependency removed while generated or test code still uses it
- tests still bootstrap the Dropwizard feature runtime

Complete integration

- Talon module added only for an integration the application uses
- Talon constructs and injects the standard client or service
- new configuration is consumed by the integration
- legacy construction is removed or explicitly retained
- tests and validation exercise the migrated feature

If any part remains: Heliport preserves required compatibility and reports the developer action instead of marking the feature complete.

Panga route tests move to Helidon @RoutingTest

Before: Dropwizard JUnit extension

```
@ExtendWith(DropwizardExtensionsSupport.class)
final class ItemEndpointTest {
    private final DirectClient client;

    ItemEndpointTest(DirectClient client) {
        this.client = client;
    }

    @SetUpRoute
    static void routes(HttpRouting.Builder routes) { ...
    }

    @Test void getItem() {
        assertThat(client.get("/v1/items/1"))...;
    }
}
```

After: Helidon routing test

```
@RoutingTest
final class ItemEndpointTest {
    private final DirectClient client;

    ItemEndpointTest(DirectClient client) {
        this.client = client;
    }

    @SetUpRoute
    static void routes(HttpRouting.Builder routes) { ...
    }

    @Test void getItem() {
        assertThat(client.get("/v1/items/1"))...;
    }
}
```

Preserve the test behavior. @RoutingTest supplies DirectClient without starting a server socket; route setup and assertions remain unchanged.

Heliport runs migration work in five ordered stages

Understand

preflight
inventory
manifest plan
repository facts

Prepare

Java 25 and tools
generated contracts
Maven prerequisites

Migrate

runtime owner waves
generated alignment
test fixtures

Converge

rerun affected work
classify remaining
work

Validate

generated sources
compile
unit tests
install

Phase and status answer different questions. The phase shows where Heliport is working or stopped. `MIGRATION_COMPLETE` means no customer-visible work remains; `MIGRATION_SUCCEEDED_WITH_REQUIRED_ACTIONS` means listed actions remain. `MIGRATION_INCOMPLETE`, `MIGRATION_BLOCKED`, and `VALIDATION_FAILED` identify other stopping conditions.

Every run records its state and reports under `.heliport`

<code>.heliport/</code>	
├─ <code>migration-report.md</code>	← start here
├─ <code>application-action-plan.json</code>	← Codex actions
├─ <code>workflow-state.json</code>	← <code>--resume</code> state
├─ <code>preflight.json</code>	← workspace checks
├─ <code>validation-report.json</code>	← validation details
├─ <code>migration-maintainer-report.md</code>	← support diagnostics
├─ <code>evidence/</code>	← technical records
├─ <code>phase-results/</code>	← phase results
├─ <code>rewrite/</code>	← transformation records

Current status lives under `.heliport/`. Keep it with the migrated files; root-level migration reports are historical context.

Start with migration-report.md for results and next steps

1 Outcome

What Heliport changed, the final status, and where the workflow finished or stopped

2 Validation

Which validation checks ran, passed, failed, or were blocked

3 Remaining work

Relevant file or code, expected result, who should act, and how to verify completion

4 Continuation

What Codex should do next and when to run --resume

Migration outcome and validation results are reported separately. A failed check remains failed until a rerun passes, even when Heliport completed substantial migration work.

Each remaining action identifies who should handle it

Responsible party	Use when	Expected next step
<code>application_owner</code>	The service team must decide business, API, security, persistence, dependency, or operational behavior.	Make the decision and implement the intended behavior.
<code>codex_app_local</code>	The repository contains enough information for Codex to make a focused change.	Ask Codex to apply the cited change and run its completion checks.
<code>heliport_product</code>	Heliport should add or improve repeatable support; the action may still authorize an application-local Codex repair.	Follow <code>repair_authority</code> : apply an authorized Codex repair now; otherwise contact Heliport. Keep the branch and <code>.heliport</code> .
<code>environment_owner</code>	Maven, repository access, credentials, network, or toolchain setup blocks progress.	Fix the prerequisite and run <code>--resume</code> .

These are exact `application-action-plan.json` values. `responsible_party` identifies long-term responsibility; `repair_authority` and `codex_app_repair_authorized` identify who can act now.

Migration status describes the current outcome and next step

Status	Meaning
MIGRATION_COMPLETE	No remaining migration work was reported to the service team.
MIGRATION_SUCCEEDED_PENDING_VALIDATION	Migration work succeeded, but one or more requested validation checks have not yet run.
MIGRATION_SUCCEEDED_WITH_REQUIRED_ACTIONS	Substantial migration completed, but listed application, Heliport, environment, or validation actions remain.
MIGRATION_INCOMPLETE	The workflow stopped before the current phase finished; review the report before resuming.
MIGRATION_BLOCKED	A reported blocker or prerequisite prevents the current phase from completing.
VALIDATION_FAILED	A required check failed, and Heliport could not identify one specific next action or a resumable stopping point.

Migration outcome and generated-source, compile, unit-test, and install results are reported separately.

Validation increases in scope from generated sources to Maven install

1

Generated sources

Generate sources and service metadata; verify that generated contracts are current.

2

Compile

Production and generated code type-check.

3

Unit tests

Migrated behavior and fixtures execute.

4

Maven install

Quality, dependency, packaging, and reactor checks run.

Migration result

The report states the migration status and lists any required actions.

Validation result

Each check retains its actual result until the corresponding command is rerun.

Migration outcome and validation are related but separate. A failed install can reveal remaining build or packaging work even after Heliport has completed most source transformations.

Resume continues from the saved .heliport state

Start with the report

```
.heliport/migration-report.md
```

It identifies why the run stopped and the next required action. Open phase results or agent status only when the report or a Heliport maintainer points to them.

Resume the migration

```
$heliport-migrate --resume
```

- resolve the prerequisite or required action named in the report
- run from the same application checkout
- Heliport rescans the current source, build, configuration, and test files
- completed transformations should make no additional edits

--resume uses the recorded state, but it reruns required checks and validation against the current files. It does not substitute an earlier passing result.

Start with the migration guide; use Slack and JIRA when needed

Heliport Migration Guide

<https://helidon.oraclecorp.com/docs/2.0.0-RC2/guides/migration/heliport-migration-guide.html>

Slack: #helidon-users

- installation and usage
- migration planning
- report and required-action questions
- early issue triage

JIRA: HLDN / Heliport

<https://jira.oci.oraclecorp.com/projects/HLDN/issues>

- create a bug, concern, or incident
- select the **Heliport** component
- describe the observed and expected behavior
- include the migration status, action ID, and validation failure when available

For the most effective diagnosis, include a link to a source-control branch containing the migrated files and the complete `.heliport/` directory.

A good migration ends with clear next steps and current validation results

- 1 Prepare Java 25 and run Heliport from the application's top-level Maven workspace.
- 2 Expect coordinated changes across the build, runtime, HTTP layer, OCI integrations, configuration, and tests.
- 3 Use `.heliport/migration-report.md` as the primary summary of results and remaining work.
- 4 Resolve required actions, resume the workflow, and address validation failures before deployment.

Heliport automates repeatable migration changes and clearly identifies the decisions and code changes that remain for the service team.

Resources

Heliport

- **Migration Guide**

<https://helidon.oraclecorp.com/docs/2.0.0-RC2/guides/migration/heliport-migration-guide.html>

- **Repository**

<https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/heliport>

- **Support:** [#helidon-users](#)

- **JIRA:** <https://jira.oci.oraclecorp.com/projects/HLDN/issues>

Select the **Heliport** component.

Helidon and Talon

- **Helidon:** <https://helidon.io/>

- **Talon RC2 documentation**

<https://helidon.oraclecorp.com/docs/2.0.0-RC2/>

- **Talon repository**

<https://devops.oci.oraclecorp.com/devops-coderepository/namespaces/axuxirvibvvo/projects/HLDN/repositories/oci-helidon>

One command starts the guided migration

```
$heliport-migrate [--resume]
                  [--branch <name>] [--commit]
                  [--maven-settings <path>]
```

Option	Use it when
<code>--resume</code>	A run stopped, or you resolved an application or environment issue reported by Heliport.
<code>--branch heliport-migration</code>	You want Heliport to create or switch to a local migration branch.
<code>--commit</code>	You want one local migration commit; requires <code>--branch</code> .
<code>--maven-settings /abs/settings.xml</code>	The application build requires an explicit Maven settings file.

Use `--dry-run` and `--validate-artifacts` only when the migration guide or Heliport support directs you to them.

Compile-time service discovery requires processor wiring

Before: Guice creates runtime bindings

```
class OrdersModule extends AbstractModule {  
    protected void configure() {  
        bind(Sender.class)  
            .to(DefaultSender.class)  
            .in(Singleton.class);  
    }  
}
```

After: Helidon generates service metadata at compile time

```
@Service.Singleton  
final class DefaultSender  
    implements Sender { ... }  
  
<annotationProcessorPaths>  
    <path>  
        <groupId>io.helidon.bundles</groupId>  
        <artifactId>helidon-bundles-apt</artifactId>  
        <version>${helidon.version}</version>  
    </path>  
</annotationProcessorPaths>
```

Compilation alone does not prove service discovery. Heliport configures `helidon-bundles-apt` so annotated services generate metadata that the Service Registry can discover at startup.

Heliport migrates Servlet, Jetty, and Netty code based on behavior

Family	Heliport can migrate	Usually requires developer review or a target design
Servlet	request/response adapters, portable listeners, and filters with direct Helidon equivalents	sessions, wrappers, body replay/hashing, connection close, proxy context, zip-bomb protection
Jetty	replace simple startup wrappers and remove unused Dropwizard bootstrap code	custom connectors, TLS reload, SNI, mTLS, HTTP/2, proxy protocol, JMX, connector-specific health
Netty	dependency cleanup when application code does not implement Netty behavior	custom channels, protocol pipelines, frame boundaries, direct Netty clients, SDK connector transport

Heliport removes a transport dependency only after verifying that source, generated code, configuration, and tests no longer use it. Remaining transport-specific behavior is reported as a required action.

Tests migrate after the production behavior they exercise

Migration sequence

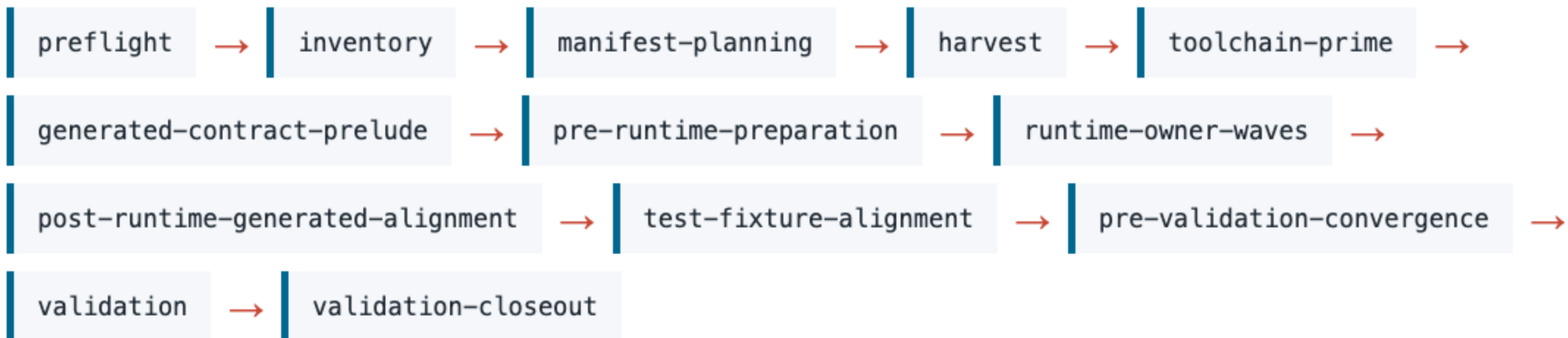
- migrate the production behavior first
- replace Dropwizard and Guice test setup
- align fixtures, generated contracts, and configuration
- rerun the original behavioral assertions

Supported examples

- health-check and admin-task tests
- generated resource and model fixtures
- Panga route tests using `@RoutingTest`
- server tests using `@ServerTest`

Keep the behavioral assertions. Changing the framework setup is not a reason to weaken or remove an assertion.

The full workflow has thirteen ordered phases



Each phase writes a result and completion check. `--resume` starts from the saved workflow state, rescans the current application files, and reruns the required checks.

Runtime migration waves run in dependency order

1. dropwizard-build
2. generated-resource-closeout
3. guice-injection
4. lifecycle-health-task
5. dropwizard-bootstrap

6. jaxrs-resource
7. request-response-carriers
8. request-response-carrier-promotions
9. jaxrs-runtime-promotions
10. jaxrs-client-closeout

11. oci-helidon
12. service-registry-metadata
13. test-migration
14. dropwizard-scaffold-closeout

After the fourteen runtime waves: Maven governance convergence checks BOM management, dependency ownership, version convergence, and pruning before validation.

A safe rerun uses saved state and current application files

1 Saved state

`workflow-state.json` records the completed phase and continuation point

2 Current files

Heliport refreshes source, build, configuration, and test facts

3 Recipes

a completed recipe should not repeat the same edits against unchanged code

4 Validation

a result changes only after its command runs again

`--resume` reruns required checks and does not treat an earlier passing result as current.

An owner frontier marks behavior Heliport cannot safely infer

Runtime

- custom Jetty/TLS/JMX
- Servlet wrappers and sessions
- proxy, gzip, streaming policy

Composition

- Guice custom scopes
- PrivateModule visibility
- AOP and dynamic factories

Platform

- custom authentication/signers/TLS
- tenancy and endpoint policy
- OCI payload/error semantics

Contracts

- custom OpenAPI templates
- public API schema meaning
- generated/handwritten integration points

Operations

- metric dimensions
- audit/redaction policy
- logging sinks and rotation

Tests

- multi-service Panga tests
- workflow drivers
- fixtures for unresolved behavior

Owner frontier is Heliport's term for behavior without a proven safe target. The action plan names the next step: service-team decision, authorized Codex repair, or Heliport enhancement.

Command and support reference

Commands

```
$heliport-migrate  
$heliport-migrate --resume  
$heliport-migrate --branch <name>  
$heliport-migrate --branch <name> --commit  
$heliport-migrate --maven-settings <path>
```

Resources

- Migration Guide: helidon.oraclecorp.com/docs/2.0.0-RC2/guides/migration/heliport-migration-guide.html
- Support: [#helidon-users](#)
- JIRA: [HLDN](#) project, [Heliport](#) component
jira.oci.oraclecorp.com/projects/HLDN/issues
- Heliport repository: [HLDN / heliport](#)
- Current status: [.heliport/migration-report.md](#)